

Sesion 8 - Integracion Continua con Jenkins

Asignatura: Herramientas y Metodos de Ingenieria del Software

Grado: Ingenieria Informatica

Universidad: Universidad de Almeria

Curso: 2025/2026

Autores:

Victor Ruz Hernandez

Jorge Ramirez Sanchez

Servidor Jenkins: <https://sesion2.spaincentral.cloudapp.azure.com/>

Repositorio: <https://github.com/uahmis2026-aultimahorasl/hmis-coches-vrh954>

1. Introduccion

En esta sesion se ha automatizado todo el pipeline de calidad construido en la **Sesion 7** (compilacion, pruebas, cobertura, analisis estatico, documentacion y analisis de vulnerabilidades) sobre un servidor **Jenkins** desplegado en una maquina virtual de Azure.

El trabajo se divide en dos fases:

1. **Job clasico tipo *Maven project*** (apartados 6.3 y 6.4 del enunciado), donde se enriquece el job creado en la sesion anterior con la publicacion de los reportes de cobertura JaCoCo, Javadoc y Maven Site.
2. **Pipeline declarativo (`Jenkinsfile`)** definido como *Pipeline as Code* en el propio job (apartado 7), que orquesta los mismos pasos como un pipeline reproducible.

2. Configuracion previa de Jenkins

2.1 Plugins instalados

Para soportar todas las acciones de la sesion se han instalado los siguientes plugins desde *Manage Jenkins* → *Plugins*:

Plugin	Uso
Pipeline	Soporte de <i>Pipeline as Code</i>
Pipeline Maven Integration	Integracion del ciclo Maven en pipelines
Git	Clonado del repositorio
JUnit	Publicacion de resultados de pruebas
JaCoCo	Reportes de cobertura (gráficas en el job)
Coverage	API moderna <code>recordCoverage</code>
HTML Publisher	Publicacion del Maven Site, Javadoc y OWASP

2.2 Herramientas globales (Tools)

En *Manage Jenkins* → *Tools* se han registrado:

- **JDK** con nombre `OpenJDK` (versión 17+).
- **Maven** con nombre `Maven-3` , instalado automaticamente desde Apache.

Estos nombres son los utilizados desde el bloque `tools { ... }` del `Jenkinsfile`.

JDK configurado:

instalaciones de JDK

instalaciones de JDK ^ Edited

+ Añadir JDK

⋮ JDK

Nombre

OpenJDK

JAVA_HOME

/usr/lib/jvm/java-21-openjdk-amd64

☒ Instalar automáticamente ?

+ Añadir un instalador

+ Añadir JDK

Maven configurado:

instalaciones de Maven

instalaciones de Maven ^ Edited

+ Añadir Maven

⋮ Maven

Nombre

Maven-3

☒ Instalar automáticamente ?

⋮ Instalar desde Apache

Versión

3.9.12

+ Añadir un instalador

3. Job Maven project - Apartados 6.3 y 6.4

3.1 Cobertura de código (apartado 6.3)

Sobre el job tipo **Maven project** creado en la sesión 7 se añade la acción posterior "**Record JaCoCo coverage report**" con los siguientes parámetros:

Campo	Valor
Path to exec files	**/target/jacoco.exec
Path to class directories	**/target/classes
Path to source directories	**/src/
Exclusions	**/test/

Acciones para ejecutar después.

Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs.

Recoger informes de cobertura JaCoCo ?

Path to exec files (e.g.: `**/target/**.exec`, `**/jacoco.exec`) Inclusions (e.g.: `**/*.class`) Exclusions (e.g.: `**/*Test*.class`)

`**/target/jacoco.exec`

Path to class directories (e.g.: `**/target/classDir`, `**/classes`)

`**/target/classes`

Path to source directories (e.g.: `**/mySourceFiles`) Inclusions (e.g.: `**/*.java`, `**/*.groovy`, `**/*.gs`) Exclusions (e.g.: `generated/**/*.*.java`)

`**/src/` `**/*.java,**/*.groovy,**/*.kt,**/*.kts` `**/test/`

☐ Disable display of source files for coverage ?

☐ Change build status according to the defined thresholds ?

☐ Always run coverage collection, even if build is FAILED or ABORTED ?

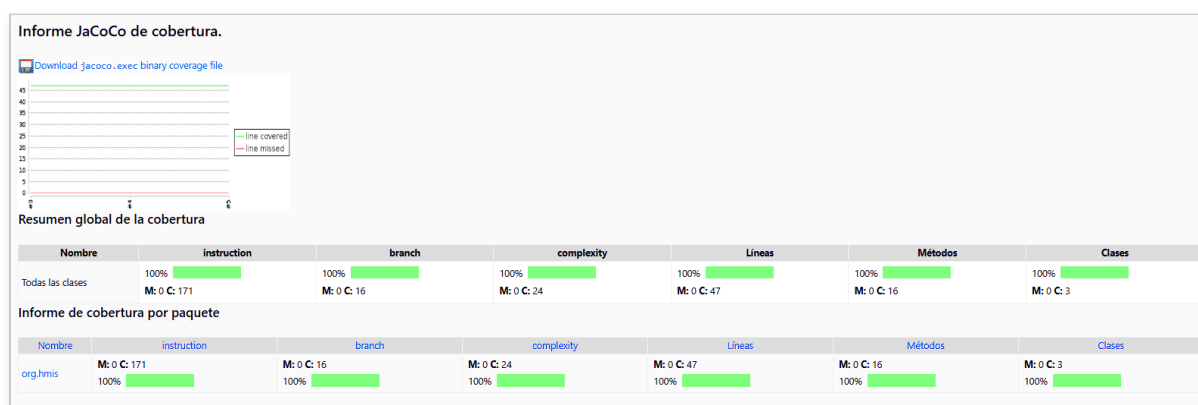
	Instruction	% Branch	% Complexity	% Líneas	% Métodos	% Clases
	0	0	0	0	0	0
	0	0	0	0	0	0

☐ Fail the build if coverage degrades more than the delta thresholds ?

	Instruction	% Branch	% Complexity	% Líneas	% Métodos	% Clases
	0	0	0	0	0	0

Los campos clave son `**/target/jacoco.exec` , `**/target/classes` , `**/src/` y la exclusion `**/test/` . Todos los umbrales se dejan a cero (desactivados), de modo que el plugin solo recoge y publica metricas sin alterar el resultado del build.

Tras lanzar el job varias veces, Jenkins agrega un *Informe JaCoCo de cobertura* a la pagina principal del job. Este informe combina la grafica de tendencia entre builds (lineas cubiertas vs. perdidas) con el resumen global y el desglose por paquete:



El proyecto **hmis-coches** mantiene el **100% de cobertura** en las seis dimensiones que reporta JaCoCo -*instruction*, *branch*, *complexity*, *lineas*, *metodos* y *clases*- sobre el unico paquete del proyecto, `org.hmis` .

3.2 Javadoc y Maven Site (apartado 6.4)

Para integrar la documentacion del proyecto en el job se modifica el campo *Goals and options* a:

```
clean verify javadoc:javadoc javadoc:aggregate site
```

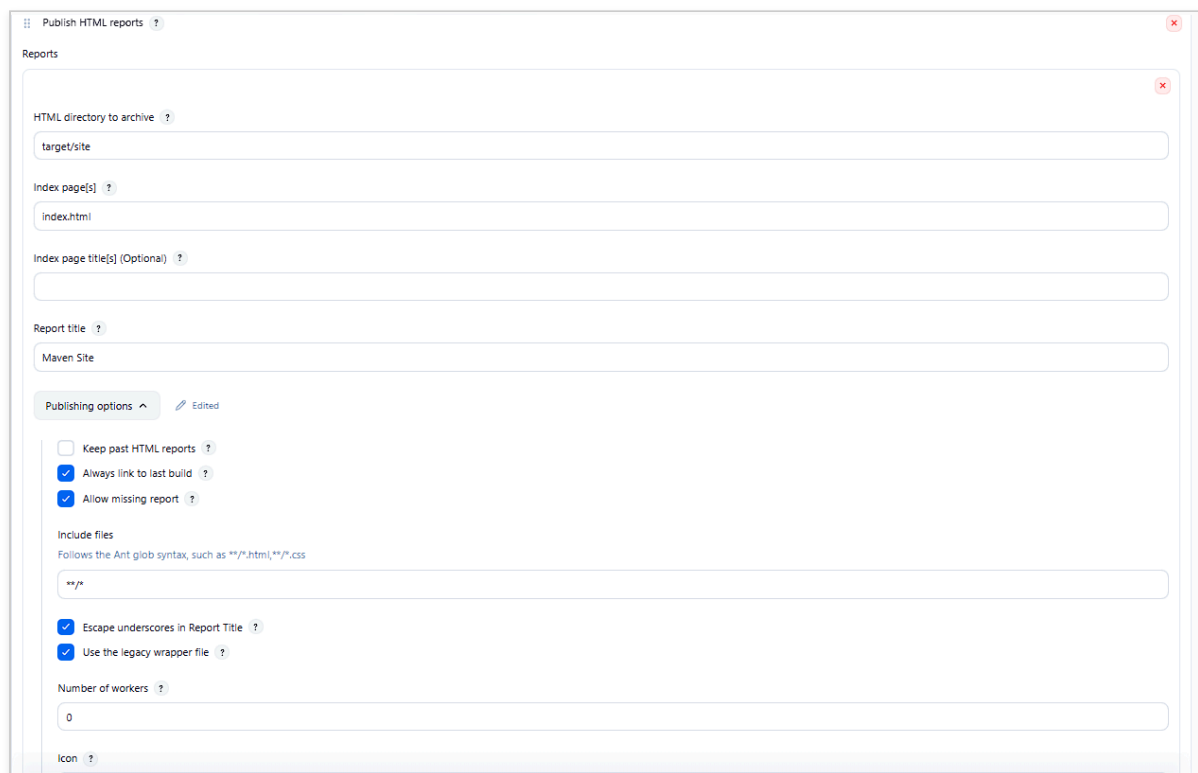
A continuacion se anaden las acciones posteriores del HTML Publisher apuntando a `target/site/es` (Maven Site) y `target/site/es/apidocs` (Javadoc), debido a que el sitio se genera en multi-idioma y el `index.html` raiz no esta directamente en `target/site/`.

Si Jenkins muestra el sitio sin estilos por la politica de seguridad del *Directory Browser*, en *Manage Jenkins* → *Script Console* se ejecuta:

```
System.setProperty("hudson.model.DirectoryBrowserSupport.CSP", "")
```

con lo que el sitio recupera CSS y JS hasta el siguiente reinicio de Jenkins.

Configuracion de las acciones posteriores del job:



The screenshot shows the 'Publish HTML reports' configuration page in Jenkins. The 'Reports' section is expanded, showing the following fields:

- HTML directory to archive**: `target/site`
- Index page[s]**: `index.html`
- Index page title[s] (Optional)**: (empty)
- Report title**: `Maven Site`

Below the 'Reports' section is the 'Publishing options' section, which is expanded. It contains the following options:

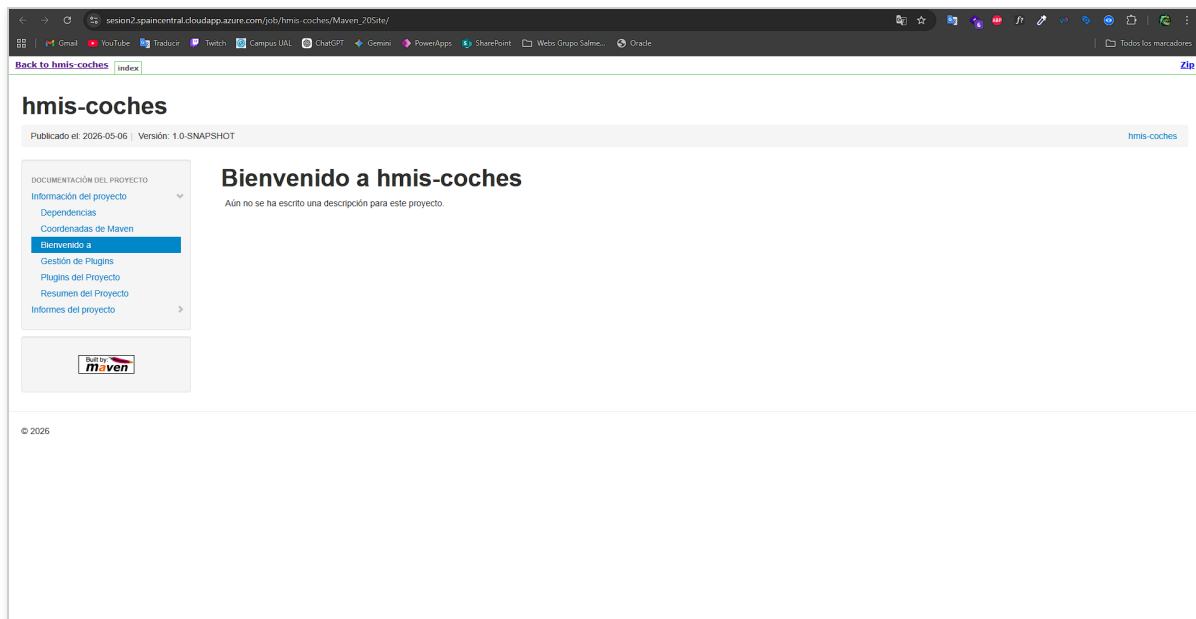
- ☐ Keep past HTML reports
- ☒ Always link to last build
- ☒ Allow missing report

Below the 'Publishing options' section is the 'Include files' section, which contains the following options:

- Follows the Ant glob syntax, such as `**/*.html,**/*.css`**: `**/*`
- ☒ Escape underscores in Report Title
- ☒ Use the legacy wrapper file

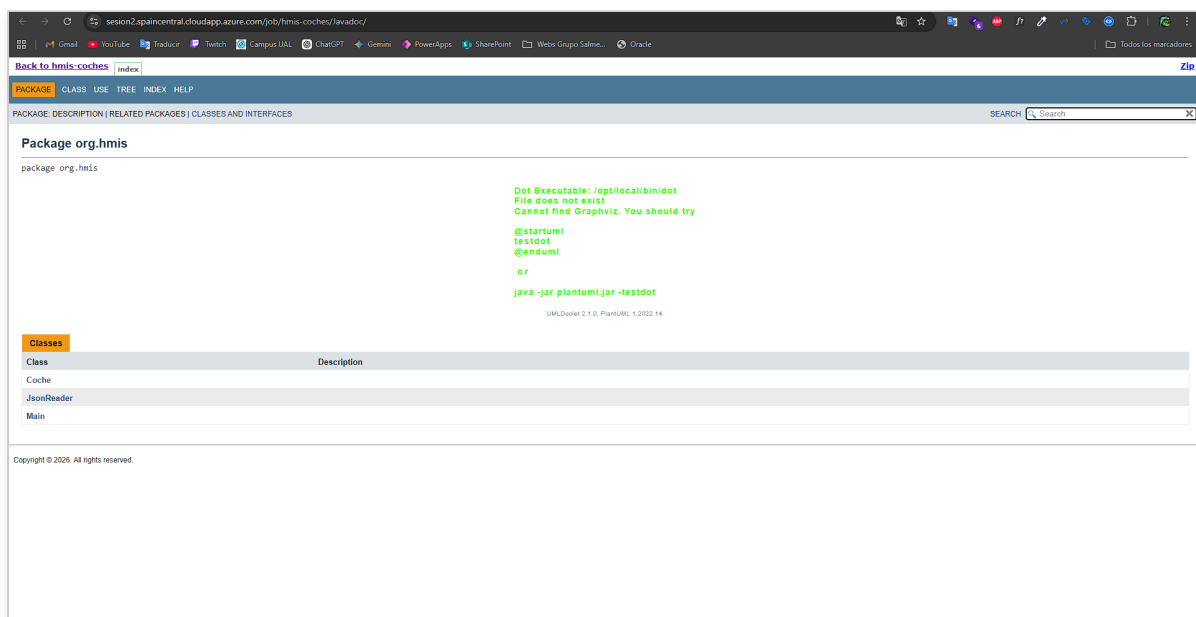
At the bottom of the page, there is a 'Number of workers' field set to `0` and an 'Icon' field.

Maven Site publicado en el job:



El Maven Site agrega en una sola vista todos los reportes generados por la sección `<reporting>` del `pom.xml`: Checkstyle, PMD, CPD, SpotBugs, JaCoCo HTML, Javadoc, JXR, Dependency-Check y los reportes estándar de información del proyecto.

Javadoc publicado en el job:



4. Pipeline declarativo - Apartado 7

4.1 Justificación

El job *Maven project* es funcional, pero acopla la definición de la build a la configuración manual de Jenkins. Esto presenta dos limitaciones:

- La build no se puede **reproducir** en otra instalacion sin replicar manualmente la configuracion del job.
- Los cambios en el ciclo de calidad **no quedan trazados en el control de versiones**.

El apartado 7 introduce la solucion: **Pipeline as Code**. Toda la cadena de calidad se describe en un script Groovy declarativo que el job de Jenkins ejecuta integro.

4.2 Estructura del Pipeline

El pipeline se organiza en cinco *stages* que reproducen la cadena de valor de la sesion 7:

```
Git fetch → Compile/Test/Package → Static Analysis → Dependency-Check → Documentation
```

4.2.1 Stage Git fetch (apartado 7.1)

```
stage('Git fetch') {
    steps {
        git branch: 'main',
            url: 'https://github.com/ualhmis2026-aultimahoras1/hmis-coches-vrh954.git'
    }
}
```

Clona la rama principal del repositorio.

4.2.2 Stage Compile, Test, Package (apartado 7.2)

```
stage('Compile, Test, Package') {
    steps {
        sh "mvn clean package"
    }
    post {
        success {
            junit '**/target/surefire-reports/TEST-*.xml'
            archiveArtifacts artifacts: '**/target/*.jar', allowEmptyArchive: true
            jacoco(execPattern: '**/target/jacoco.exec', ...)
            recordCoverage(tools: [[parser: 'JACOCO',
                pattern: '**/target/site/jacoco/jacoco.xml']])
        }
    }
}
```

Este stage cumple cuatro funciones simultaneamente:

1. Compilar el proyecto y ejecutar la bateria de pruebas JUnit 5.
2. Empaquetar el `.jar` resultante y archivarlo como artefacto del build.
3. Publicar los resultados de las pruebas con el plugin **JUnit**.
4. Publicar la cobertura tanto con el plugin clasico **JaCoCo** (graficas historicas en la pagina del job) como con el plugin moderno **Coverage** (reporte detallado por clase). Se utiliza la sintaxis nueva `recordCoverage` ; el antiguo `publishCoverage / jacocoAdapter` ha quedado deprecado en versiones recientes.

4.2.3 Stage Static Analysis (apartado 7.3)

```
stage('Static Analysis') {  
    steps {  
        sh "mvn site"  
    }  
}
```

`mvn site` regenera todos los reportes declarados en la seccion `<reporting>` del `pom.xml` : Checkstyle, PMD, CPD, SpotBugs, JaCoCo HTML, JXR y Javadoc. Los reportes resultantes se incrustan en el sitio del proyecto y se publican posteriormente desde el stage *Documentation* mediante HTML Publisher, evitando depender del plugin **Warnings Next Generation** (no instalado en la instancia de Jenkins utilizada).

4.2.4 Stage Dependency-Check (apartado 7.4)

```
stage('Dependency-Check') {  
    steps {  
        sh "mvn org.owasp:dependency-check-maven:check"  
    }  
}
```

El plugin **OWASP Dependency-Check** descarga la base de datos NVD (la primera ejecucion tarda 20-30 min y la BBDD se cachea para builds posteriores) y produce los informes `dependency-check-report.html` y `.xml` . El reporte HTML se publica en el stage final mediante HTML Publisher.

4.2.5 Stage Documentation (apartado 7.5)

```
stage('Documentation') {  
    steps {  
        sh "mvn javadoc:javadoc javadoc:aggregate"  
    }  
    post {  
        success {  
            publishHTML(target: [reportName: 'Maven Site',  
                                reportDir: 'target/site/es', reportFiles: 'index.html', ...])  
            publishHTML(target: [reportName: 'Javadoc',  
                                reportDir: 'target/site/es/apidocs', reportFiles: 'index.html', ...])  
            publishHTML(target: [reportName: 'OWASP Dependency-Check',  
                                reportDir: 'target/site/es',  
                                reportFiles: 'dependency-check-report.html', ...])  
        }  
    }  
}
```

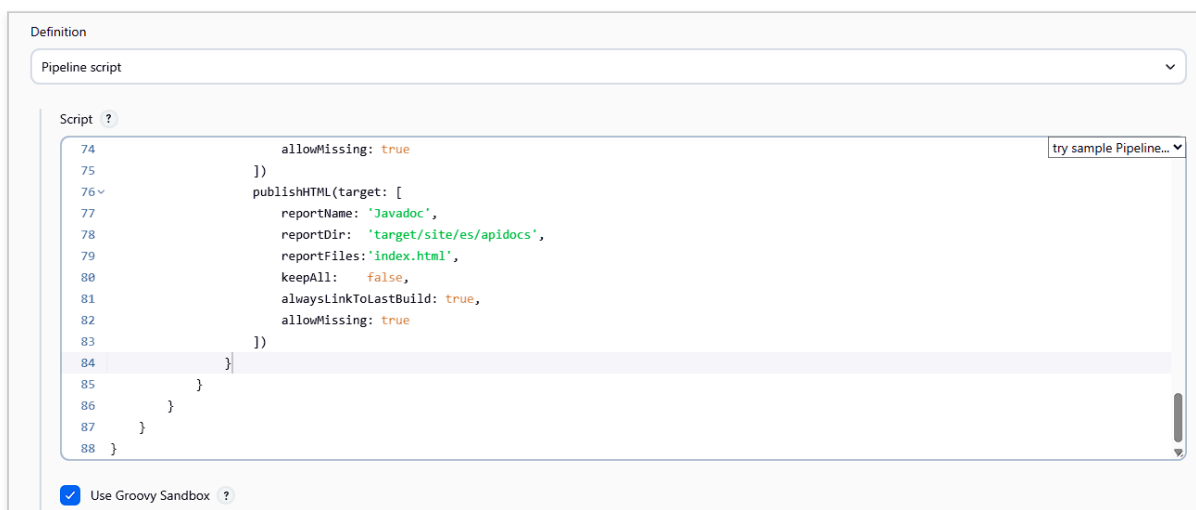
Genera el Javadoc agregado y publica tres reportes HTML como enlaces permanentes en el lateral del job: **Maven Site** (indice del sitio Maven con todos los reportes de calidad), **Javadoc** (documentacion API enriquecida con UMLDoclet) y **OWASP Dependency-Check** (informe de vulnerabilidades). Como el `<site>` esta configurado en multi-idioma, los paths apuntan a `target/site/es/...` en lugar de a la raiz del site.

4.3 Configuración del job Pipeline

En Jenkins se ha creado un nuevo *Item* tipo **Pipeline** llamado `hmis-coches-pipeline` con la siguiente definición:

Campo	Valor
Definition	Pipeline script (definición <i>inline</i> en Jenkins)
Use Groovy Sandbox	Activado
Script	El contenido del Anexo (sección 8)

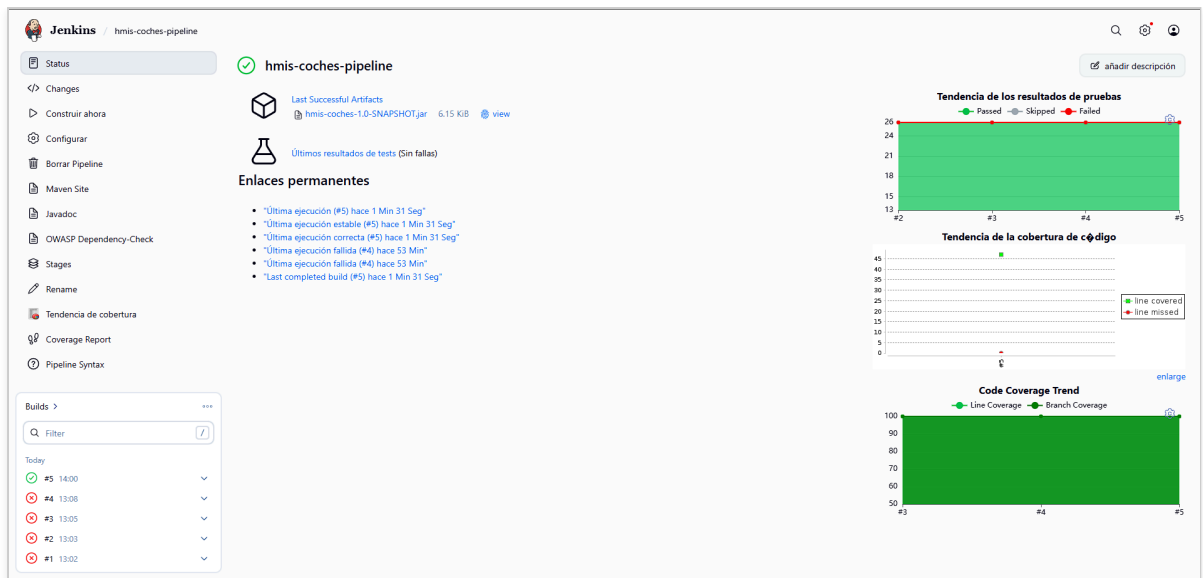
Se ha optado por **Pipeline script** (inline) en lugar de *Pipeline script from SCM* para mantener la definición del pipeline aislada del repositorio durante esta práctica.



5. Resultados de la ejecución del pipeline

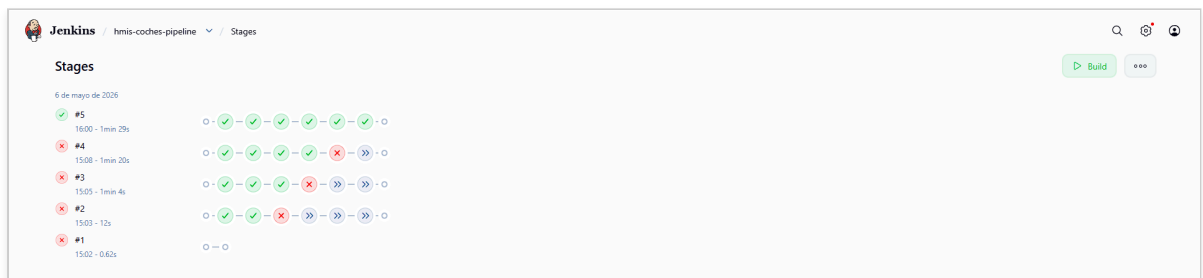
5.1 Vista general del job

La página principal del job ofrece una visión consolidada con artefactos generados, último resultado de tests, gráficas de tendencia (resultados de pruebas, cobertura de código y *Code Coverage Trend*) y los enlaces permanentes a los tres reportes HTML publicados (Maven Site, Javadoc, OWASP Dependency-Check).

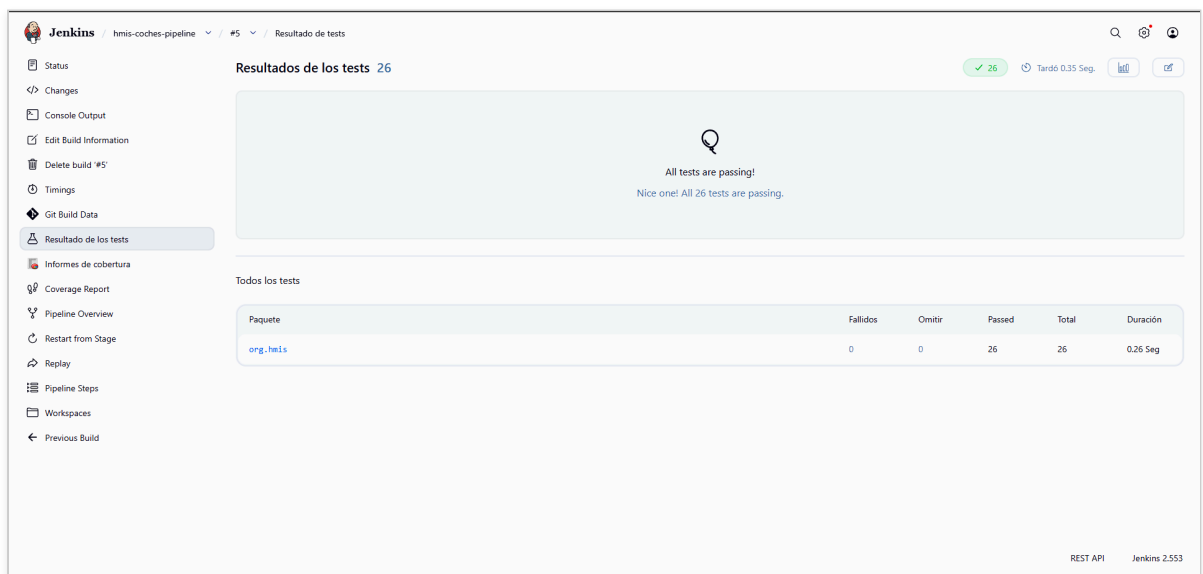


5.2 Stage View

Jenkins representa la ejecución de los stages como una *Stage View*, permitiendo identificar de un vistazo el estado y la duración de cada fase.

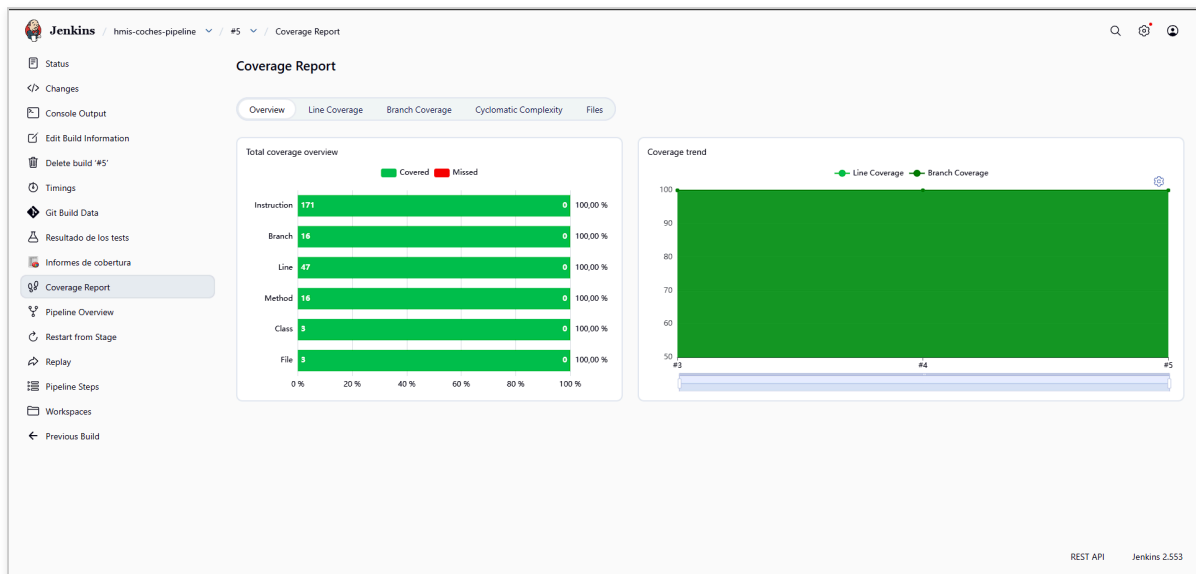


5.3 Pruebas JUnit



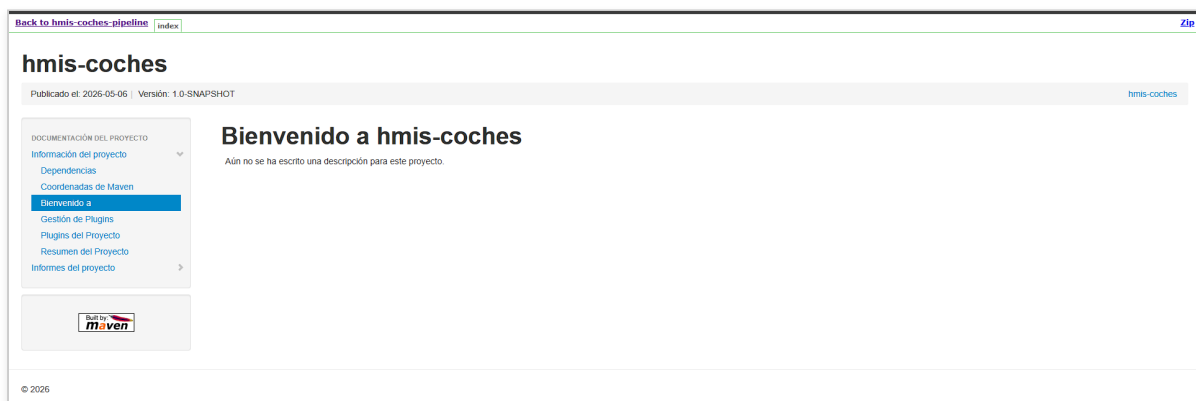
Las **26 pruebas** del proyecto pasan en verde, cubriendo todos los casos de `Coche`, `JsonReader` y `Main`.

5.4 Cobertura del codigo



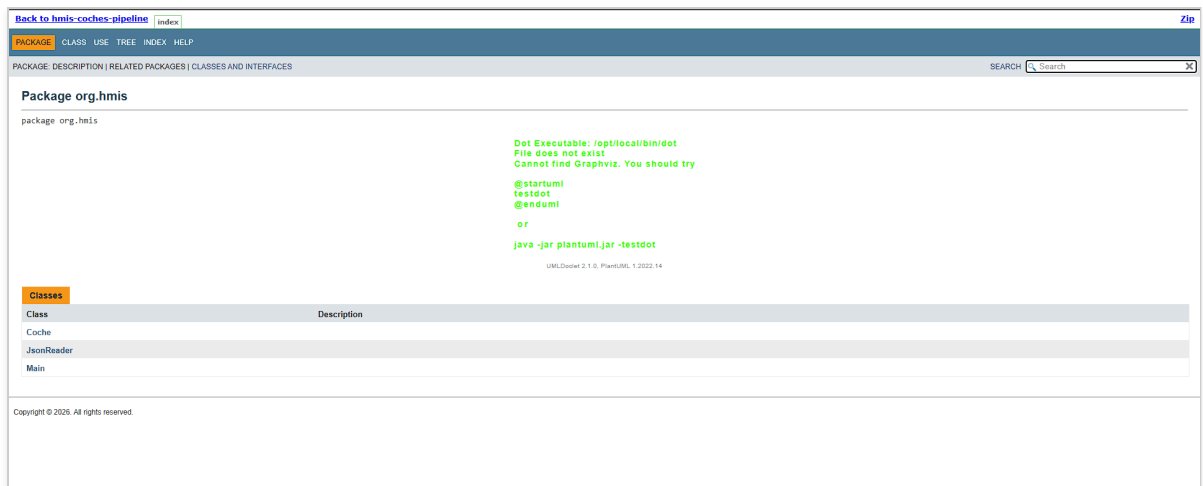
El plugin **Coverage** muestra la cobertura agregada al **100%** en modulo, paquete, clase, metodo, linea, rama e instruccion, coherente con la sesion 7.

5.5 Maven Site

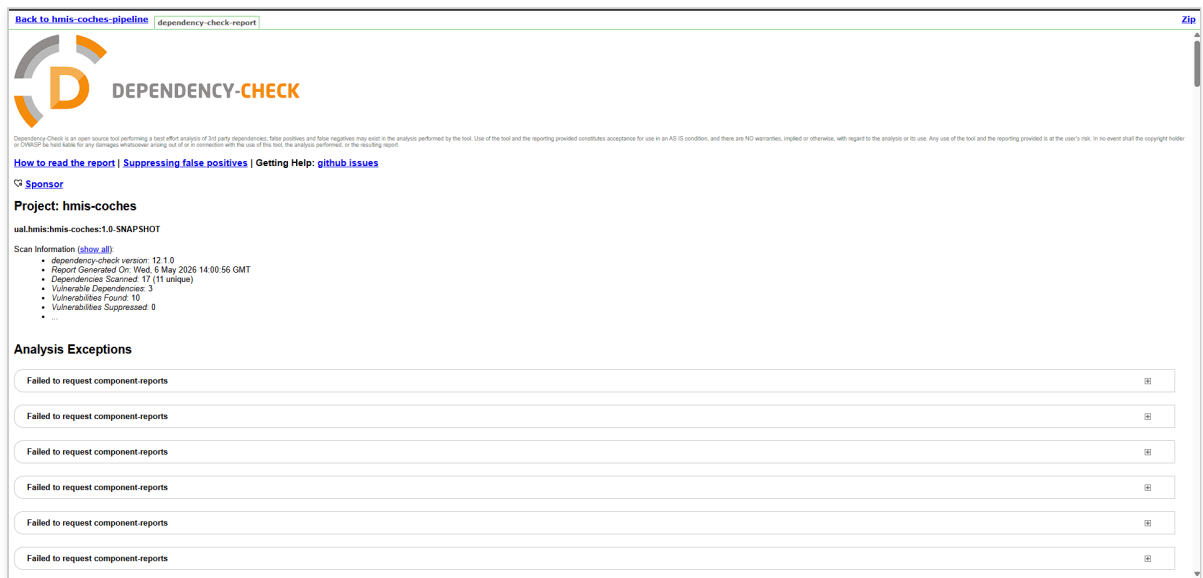


El sitio Maven publicado por HTML Publisher contiene la totalidad de los reportes de calidad del proyecto: informacion del proyecto, dependencias, Checkstyle, PMD, CPD, SpotBugs, JXR, JaCoCo HTML y Javadoc.

5.6 Javadoc



5.7 OWASP Dependency-Check



El reporte detecta las vulnerabilidades introducidas a proposito en commons-collections4 4.0 (CVE-2015-6420) y en log4j-core 2.15.0 (CVE-2021-45046, CVE-2021-44832, CVE-2021-45105 y CVEs recientes 2025/2026), validando que el pipeline identifica correctamente las dependencias inseguras desde Jenkins.

5.8 Console Output

```
[WARNING] Note: Add UML to /var/lib/jenkins/workspace/hmis-coches-pipeline/target/site/apidocs/org/hmis/JsonReader.html...
[WARNING] 21 warnings
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 7.498 s
[INFO] Finished at: 2026-05-06T14:01:41Z
[INFO] -----
Post stage
[Pipeline] publishHTML
[htmlpublisher] Archiving HTML reports...
[htmlpublisher] Archiving at PROJECT level /var/lib/jenkins/workspace/hmis-coches-pipeline/target/site/es to Maven_20Site
[htmlpublisher] Copying recursive using current thread
[Pipeline] publishHTML
[htmlpublisher] Archiving HTML reports...
[htmlpublisher] Archiving at PROJECT level /var/lib/jenkins/workspace/hmis-coches-pipeline/target/site/es/apidocs to Javadoc
[htmlpublisher] Copying recursive using current thread
[Pipeline] publishHTML
[htmlpublisher] Archiving HTML reports...
[htmlpublisher] Archiving at PROJECT level /var/lib/jenkins/workspace/hmis-coches-pipeline/target/site/es to OWASP_20Dependency-Check
[htmlpublisher] Copying recursive using current thread
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

El log de la ejecucion termina con `Finished: SUCCESS` , confirmando que los cinco stages (Git fetch, Compile/Test/Package, Static Analysis, Dependency-Check, Documentation) se han completado correctamente.

6. Conclusiones

La sesion ha completado la cadena de calidad iniciada en la sesion 7 transformandola de una ejecucion manual en una build **continua, reproducible y trazable**.

1. **De configuracion manual a Pipeline as Code:** la build queda definida en el script Groovy del job, eliminando la dependencia del clic-a-clic de configuracion local de Jenkins.
2. **Una unica fuente de verdad:** el mismo `pom.xml` y la misma configuracion de plugins de la sesion 7 se ejecutan ahora desde Jenkins sin duplicar configuracion.
3. **Visibilidad continua:** Jenkins agrega graficas historicas de tests y cobertura, y publica los reportes HTML detallados de Checkstyle, PMD, SpotBugs, OWASP y Javadoc, lo que permite detectar regresiones tan pronto como se introducen.
4. **Reproducibilidad:** cualquier desarrollador puede levantar un Jenkins limpio, copiar el script del Anexo y reproducir la build completa sin mas intervencion manual.

El siguiente paso natural -no incluido en el alcance de esta practica- seria mover el script Groovy a un `Jenkinsfile` versionado en el propio repositorio (modalidad *Pipeline script from SCM*), o incluso utilizar un *Multibranch Pipeline*, que crea automaticamente un job por rama.

7. Repositorio y entregables

- **Repositorio:** <https://github.com/ualhmis2026-aultimahorasl/hmis-coches-vrh954>
- **Servidor Jenkins:** <https://sesion2.spaincentral.cloudapp.azure.com/>
- **Job Maven project:** `hmis-coches`
- **Job Pipeline:** `hmis-coches-pipeline`

8. Anexo - Script del Pipeline

```

pipeline {
    agent any

    tools {
        maven "Maven-3"
        jdk "OpenJDK"
    }

    stages {

        stage('Git fetch') {
            steps {
                git branch: 'main',
                    url: 'https://github.com/ualhmis2026-a ultimahoras1/hmis-coches-vrh954.git'
            }
        }

        stage('Compile, Test, Package') {
            steps {
                sh "mvn clean package"
            }
            post {
                success {
                    junit '**/target/surefire-reports/TEST-*.xml'
                    archiveArtifacts artifacts: '**/target/*.jar', allowEmptyArchive: true
                    jacoco(
                        execPattern: '**/target/jacoco.exec',
                        classPattern: '**/target/classes',
                        sourcePattern: '**/src/',
                        exclusionPattern: '**/test/'
                    )
                    recordCoverage(tools: [[parser: 'JACOCO',
                        pattern: '**/target/site/jacoco/jacoco.xml']])
                }
            }
        }

        stage('Static Analysis') {
            steps {
                sh "mvn site"
            }
        }

        stage('Dependency-Check') {
            steps {
                sh "mvn org.owasp:dependency-check-maven:check"
            }
        }

        stage('Documentation') {
            steps {
                sh "mvn javadoc:javadoc javadoc:aggregate"
            }
            post {
                success {
                    publishHTML(target: [
                        reportName: 'Maven Site',
                        reportDir: 'target/site/es',
                        reportFiles: 'index.html',
                        keepAll: false,
                    ])
                }
            }
        }
    }
}

```

```
        alwaysLinkToLastBuild: true,  
        allowMissing: true  
    })  
    publishHTML(target: [  
        reportName: 'Javadoc',  
        reportDir: 'target/site/es/apidocs',  
        reportFiles: 'index.html',  
        keepAll: false,  
        alwaysLinkToLastBuild: true,  
        allowMissing: true  
    ])  
    publishHTML(target: [  
        reportName: 'OWASP Dependency-Check',  
        reportDir: 'target/site/es',  
        reportFiles: 'dependency-check-report.html',  
        keepAll: false,  
        alwaysLinkToLastBuild: true,  
        allowMissing: true  
    ])  
    }  
    }  
    }  
    }  
}
```